🏛 GLA UNIVERSITY, MATHURA — DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

HPC Masterclass | Day 4 Pre-Assessment Question Paper

Course: B.Tech CSE 7th Sem — High Performance Computing Masterclass
Module: Day 4 Baseline Readiness — Profiling Concepts, Processor Pipelines, Memory Subsystems & Cluster Orchestration
Duration: 60 Minutes | **Total Questions**: 50 MCQs | **Maximum Marks**: 50

Student Name: _____ **Roll Number**: _____ **Section**: _____

📝 Instructions for Candidates

1. All questions are compulsory. Each question carries exactly 1 mark.
2. Select the single best answer for each question.
3. No negative marking.

Q01. What is the primary operational difference between 'Profiling' and 'Debugging' in software engineering?

- ▢ **(a)** Profiling finds syntax errors; debugging finds logic errors.
- ▢ **(b)** Debugging investigates functional correctness and crashes; profiling measures dynamic resource consumption (execution time, memory bandwidth, cache misses) to identify performance bottlenecks.
- ▢ **(c)** Profiling only works on Windows; debugging only works on Linux.
- ▢ **(d)** Debugging requires recompiling the operating system kernel.

Q02. What is the primary drawback of using source-code 'Instrumentation' (e.g., injecting manual timers around every function) compared to statistical sampling?

- ▢ **(a)** Instrumentation cannot be compiled by GCC.
- ▢ **(b)** Probe effect / overhead: inserting instrumentation code alters execution timing, pollutes instruction caches, and distorts the true performance characteristics of the application.
- ▢ **(c)** Instrumentation makes the application run 100 times faster.
- ▢ **(d)** Instrumentation only records network bandwidth.

Q03. What does the metric 'Instructions Per Cycle' (IPC) signify when evaluating CPU application efficiency?

- ▢ **(a)** The number of Python files executed per second.
- ▢ **(b)** The average number of machine instructions retired by the processor execution pipeline during each clock cycle.
- ▢ **(c)** The rotational speed of the cooling fans.
- ▢ **(d)** The total amount of memory allocated on the heap.

Q04. According to Amdahl's Law, what limits the maximum theoretical speedup of an application when adding an infinite number of parallel processor cores?

- ▢ **(a)** The maximum capacity of the power supply.
- ▢ **(b)** The strictly sequential (serial) fraction of the algorithm that cannot be parallelized.
- ▢ **(c)** The physical length of the Ethernet network cables.
- ▢ **(d)** The clock frequency of the system real-time clock.

Q05. In software profiling, what is a performance 'Hotspot'?

- ▢ **(a)** A Wi-Fi access point in the server room.
- ▢ **(b)** A localized function or code section that consumes a disproportionately large fraction of the total execution time or processor cycles.
- ▢ **(c)** A CPU core that has exceeded its thermal limit.
- ▢ **(d)** A variable that stores sensitive user passwords.

Q06. In a profiler call tree, what is the difference between 'Self Time' and 'Inclusive Time' (Cumulative Time)?

- ▢ **(a)** Self time includes compiler time; inclusive time does not.
- ▢ **(b)** Self time measures time spent solely within the instructions of that specific function; inclusive time includes self time plus the time spent in all subroutines called by that function.
- ▢ **(c)** Inclusive time is measured in seconds; self time is measured in bytes.
- ▢ **(d)** Self time only counts idle wait states.

Q07. Why is statistical sampling profiling considered superior to full function-entry/exit tracing for production HPC workloads?

- ▢ **(a)** Statistical sampling produces gigabytes of text files per second.
- ▢ **(b)** Statistical sampling introduces minimal runtime overhead (< 2%), does not require binary recompilation, and avoids massive trace file growth.
- ▢ **(c)** Statistical sampling only records the exit code of the program.
- ▢ **(d)** Tracing cannot record floating-point operations.

Q08. What is the difference between 'CPU Time' and 'Wall-Clock Time' reported by performance profiling tools?

- ▢ **(a)** CPU time is measured by a physical clock on the wall; wall-clock time is measured by the CPU.
- ▢ **(b)** CPU Time is the total active execution time spent by all processor cores executing the process's instructions; Wall-Clock Time is the actual real-world elapsed time between process start and completion.
- ▢ **(c)** Wall-clock time is always greater than CPU time on multi-core systems.
- ▢ **(d)** CPU time does not count time spent in user space.

Q09. What problem arises when profiling multi-threaded applications if thread synchronizations (such as mutex locks or barriers) are frequent?

- ▢ **(a)** The CPU frequency drops to zero.
- ▢ **(b)** Threads spend significant wall-clock time in idle wait states (waiting for locks/barriers), which may look like low CPU utilization rather than an active computation hotspot.
- ▢ **(c)** The operating system automatically terminates all threads.

- ▢ **(d)** The profiler crashes with a syntax error.

Q10. Why must an executable be compiled with debug symbols (e.g., `-g` flag in GCC/ICX) when preparing for in-depth profiling?

- ▢ **(a)** Without `-g`, the executable cannot run on 64-bit systems.
- ▢ **(b)** Debug symbols provide the mapping (DWARF/PDB tables) between machine instruction addresses and original source-code filenames and line numbers, enabling line-by-line profiling.
- ▢ **(c)** Debug symbols automatically disable all compiler optimizations.
- ▢ **(d)** Debug symbols allow the compiler to send data over the internet.

Q11. In modern superscalar processors, what is the role of the 'Out-of-Order Execution' (OoO) engine?

- ▢ **(a)** Executing instructions in random order to scramble memory for security.
- ▢ **(b)** Executing instructions as soon as their data dependencies are ready and execution units are free, rather than waiting in strict program order, hiding memory access delays.
- ▢ **(c)** Reversing the flow of electrical current through the silicon gates.
- ▢ **(d)** Running instructions backwards to implement undo operations.

Q12. What happens when a CPU branch predictor incorrectly predicts the outcome of a conditional branch instruction?

- ▢ **(a)** The processor permanently halts and requires a hardware reboot.
- ▢ **(b)** Branch Misprediction Penalty: all speculatively executed uOps must be discarded (flushed from the pipeline), and instruction fetching must restart from the correct target address, wasting 15-20+ clock cycles.
- ▢ **(c)** The program automatically branches to line 0 of the code.
- ▢ **(d)** The operating system switches to 16-bit real mode.

Q13. What is the purpose of the 'Reorder Buffer' (ROB) in an out-of-order processor architecture?

- ▢ **(a)** Sorting disk sectors for hard drive defragmentation.
- ▢ **(b)** Ensuring that instructions that executed out-of-order commit their architectural register and memory modifications in strict original program order, preserving precise exceptions.
- ▢ **(c)** Reordering print statements in Python programs.
- ▢ **(d)** Compressing executable files in system RAM.

Q14. What is 'Register Renaming' in superscalar processors, and what performance hazard does it eliminate?

- ▢ **(a)** Changing variable names in source code to make them shorter.
- ▢ **(b)** Mapping architectural registers to a larger pool of physical registers to eliminate false dependencies: Write-After-Read (WAR) and Write-After-Write (WAW) hazards.
- ▢ **(c)** Renaming thread identifiers in the Linux scheduler.
- ▢ **(d)** Converting floating point numbers to integers.

Q15. What does a Fused Multiply-Add (FMA) hardware instruction compute in a single clock cycle?

- ▢ **(a)** $d = a + b + c$
- ▢ **(b)** $d = (a \times b) + c$

- ▢ **(c)** $d = (a/b) - c$
- ▢ **(d)** $d = \sqrt{a^2 + b^2}$

Q16. What is 'Simultaneous Multi-Threading' (SMT), known commercially as Intel Hyper-Threading?

- ▢ **(a)** Placing two distinct CPU motherboards inside a single chassis.
- ▢ **(b)** Exposing two logical execution contexts (architectural register states) per physical core, sharing execution units and caches to interleave execution during memory stalls.
- ▢ **(c)** Running two copies of the operating system simultaneously.
- ▢ **(d)** Executing code on both the CPU and GPU at the same time.

Q17. What is 'Store-to-Load Forwarding' in a CPU load-store execution unit?

- ▢ **(a)** Uploading files to cloud storage before reading them.
- ▢ **(b)** Forwarding data directly from an active store buffer to a subsequent load instruction requesting the same memory address, bypassing cache and DRAM access entirely.
- ▢ **(c)** Converting memory stores into network packets.
- ▢ **(d)** Storing CPU registers to the stack on function calls.

Q18. What causes 'Thermal Throttling' in high-performance computing processors?

- ▢ **(a)** The processor temperature drops below 0 degrees Celsius.
- ▢ **(b)** The silicon die temperature reaches the maximum junction threshold ($T_j\max$), triggering hardware thermal watchdogs that dynamically downclock CPU frequency to prevent physical damage.
- ▢ **(c)** The operating system runs out of disk storage.
- ▢ **(d)** The liquid cooling tubes become transparent.

Q19. How wide are Intel AVX2 vector registers (YMM0 to YMM15), and how many 64-bit double-precision floats can one register hold?

- ▢ **(a)** 128 bits wide, holding 2 doubles.
- ▢ **(b)** 256 bits wide, holding 4 double-precision (FP64) floats.
- ▢ **(c)** 512 bits wide, holding 8 doubles.
- ▢ **(d)** 64 bits wide, holding 1 double.

Q20. What is 'Dynamic Voltage and Frequency Scaling' (DVFS) in modern microprocessors?

- ▢ **(a)** A technique to convert alternating current (AC) to direct current (DC).
- ▢ **(b)** A hardware power management feature that dynamically adjusts operating clock frequency and core supply voltage based on computational workload demands and thermal limits.
- ▢ **(c)** A software algorithm for resizing JPEG images.
- ▢ **(d)** A compiler optimization for unrolling loops.

Q21. What is the typical memory hierarchy capacity and latency progression on modern Intel Xeon Scalable processors?

- ▢ **(a)** L1 is slowest and largest; DRAM is fastest and smallest.

- ▢ **(b)** L1 (32-48 KB, ~1 ns / 4 cycles) -> L2 (1-2 MB, ~3-4 ns / 14 cycles) -> L3/LLC (shared ~1.5-2.5 MB/core, ~12-20 ns / 40-50 cycles) -> DRAM (~60-90 ns / 200+ cycles).
- ▢ **(c)** All cache levels have identical capacity and latency.
- ▢ **(d)** DRAM is on-chip; L1 cache is on an external PCIe card.

Q22. What is 'Spatial Locality' in computer memory access patterns?

- ▢ **(a)** Accessing the exact same memory address repeatedly over time.
- ▢ **(b)** The principle that if a particular memory location is accessed, nearby contiguous memory locations are likely to be accessed soon thereafter.
- ▢ **(c)** Storing data on satellites in space.
- ▢ **(d)** Distributing files across multiple hard drives.

Q23. What is 'Temporal Locality' in software execution?

- ▢ **(a)** The principle that data or instructions accessed recently are likely to be accessed again in the near future.
- ▢ **(b)** Accessing data only on weekends.
- ▢ **(c)** Measuring memory transfer speeds using a stopwatch.
- ▢ **(d)** Converting memory addresses into timestamps.

Q24. What occurs during a CPU 'Translation Lookaside Buffer' (TLB) miss?

- ▢ **(a)** The computer converts English text to Spanish.
- ▢ **(b)** The hardware MMU must perform a high-latency 'page table walk' across multi-level page tables in main memory to resolve the virtual-to-physical address translation.
- ▢ **(c)** The CPU registers are wiped to zero.
- ▢ **(d)** The process terminates immediately with an error.

Q25. How do Linux 'Hugepages' (e.g., 2MB or 1GB pages instead of default 4KB pages) benefit high-throughput HPC workloads?

- ▢ **(a)** They allow text files to have larger font sizes.
- ▢ **(b)** They drastically increase the total physical memory range covered by a fixed number of TLB entries, slashing TLB misses by orders of magnitude for large data structures.
- ▢ **(c)** They eliminate the need for system RAM.
- ▢ **(d)** They prevent disk drives from filling up.

Q26. What is the function of CPU 'Hardware Data Prefetchers' (e.g., Stream Prefetcher)?

- ▢ **(a)** Downloading software updates automatically from the internet.
- ▢ **(b)** Detecting sequential or strided memory access patterns and speculatively loading anticipated cache lines from DRAM into L2/L1 cache before the CPU execution core explicitly requests them.
- ▢ **(c)** Writing random numbers into empty RAM sectors.
- ▢ **(d)** Checking for syntax errors in Python scripts.

Q27. In the MESI cache coherence protocol, what does the 'I' (Invalid) state indicate?

- ▢ **(a)** The processor is idle.

- ▢ **(b)** The cache line does not contain valid data, meaning any read or write to this address by the core will result in a cache miss requiring a fetch from LLC or DRAM.
- ▢ **(c)** The memory address is corrupt.
- ▢ **(d)** The cache line is exclusively owned and modified.

Q28. Why does traversing a two-dimensional C/C++ array column-by-column (strided) perform significantly slower than row-by-row?

- ▢ **(a)** Because C/C++ uses column-major order like Fortran.
- ▢ **(b)** Because C/C++ stores 2D arrays in row-major order; column-by-column traversal jumps by N elements between consecutive accesses, causing a cache miss on virtually every element and defeating prefetchers.
- ▢ **(c)** Because compiler optimization flags only apply to rows.
- ▢ **(d)** Because column traversal disables CPU vector instructions.

Q29. What is peak theoretical memory bandwidth for a server featuring an 8-channel DDR5-4800 memory subsystem?

- ▢ **(a)** 38.4 GB/s
- ▢ **(b)** 307.2 GB/s
- ▢ **(c)** 1000 GB/s
- ▢ **(d)** 4.8 GB/s

Q30. What is the primary architectural trade-off of High Bandwidth Memory (HBM) compared to standard DDR5 DIMMs?

- ▢ **(a)** HBM is slower but much cheaper.
- ▢ **(b)** HBM delivers massively higher bandwidth (2-3+ TB/s vs ~300 GB/s) and energy efficiency via thousands of wide interposer connections, but has lower maximum capacity and cannot be upgraded via motherboard slots.
- ▢ **(c)** HBM cannot store floating-point numbers.
- ▢ **(d)** HBM requires liquid nitrogen cooling at all times.

Q31. In the Message Passing Interface (MPI), what is the difference between 'MPI_Send' (blocking) and 'MPI_Isend' (non-blocking)?

- ▢ **(a)** MPI_Send sends numbers; MPI_Isend sends text.
- ▢ **(b)** MPI_Send blocks the calling process until the send buffer can be safely modified; MPI_Isend returns immediately and returns a request handle, allowing computation to overlap with message transit.
- ▢ **(c)** MPI_Isend is only available on single-core laptops.
- ▢ **(d)** MPI_Send requires root privileges.

Q32. In an MPI communicator of N processes, what does 'MPI_Allgather' perform?

- ▢ **(a)** It sums all numbers and leaves the result only on rank 0.
- ▢ **(b)** Each process contributes a chunk of data, and all chunks are concatenated such that every process in the communicator receives the complete combined dataset.
- ▢ **(c)** It deletes all processes except rank 0.
- ▢ **(d)** It broadcasts an identical message from root to all.

Q33. What is 'Bisection Bandwidth' in HPC network topologies?

- ▢ **(a)** The maximum bandwidth of a single USB port.
- ▢ **(b)** The minimum data transfer capacity between two equal halves of the network when the network is partitioned in the worst possible way.
- ▢ **(c)** The speed of light in optical fiber.
- ▢ **(d)** The frequency of the cluster master clock.

Q34. Why is a non-blocking Fat-Tree network topology widely deployed in HPC and AI supercomputers?

- ▢ **(a)** It looks like a pine tree in datacenter blueprints.
- ▢ **(b)** It provides full bisection bandwidth, guaranteeing that any node can communicate with any other node at full link line-rate simultaneously without cross-switch oversubscription.
- ▢ **(c)** It requires only 1 network switch for 10,000 servers.
- ▢ **(d)** It operates without physical cabling.

Q35. What does RDMA (Remote Direct Memory Access) over Converged Ethernet (RoCE v2) eliminate compared to standard TCP/IP networking?

- ▢ **(a)** It eliminates the need for Ethernet switches.
- ▢ **(b)** It eliminates operating system kernel involvement, intermediate socket buffer copies, and CPU interrupt handling, performing zero-copy memory transfers directly across network adapters.
- ▢ **(c)** It eliminates physical Ethernet MAC addresses.
- ▢ **(d)** It replaces fiber-optic cables with copper wires.

Q36. What is the primary role of Linux Control Groups (cgroups) in cluster workload management like Slurm?

- ▢ **(a)** Encrypting student source code.
- ▢ **(b)** Enforcing strict hardware resource limits (constraining CPU cores, physical RAM, and GPU devices) to prevent user jobs from interfering with each other or crashing host nodes.
- ▢ **(c)** Managing user graphical user interfaces.
- ▢ **(d)** Converting shell scripts to C++ code.

Q37. In Slurm job scheduling, what does the 'Backfill Scheduler' accomplish?

- ▢ **(a)** Deleting old completed jobs from the database.
- ▢ **(b)** Starting smaller, lower-priority jobs on idle nodes ahead of time, provided they will complete before higher-priority pending jobs are scheduled to begin, maximizing cluster utilization.
- ▢ **(c)** Re-running failed jobs with lower memory requests.
- ▢ **(d)** Backing up node hard drives to magnetic tape.

Q38. What is the function of the Slurm daemon 'slurmdbd'?

- ▢ **(a)** Running Python machine learning models on GPU nodes.
- ▢ **(b)** Managing the persistent Slurm database (MySQL/MariaDB) to store accounting records, fair-share usage history, user allocations, and job completion telemetry.
- ▢ **(c)** Monitoring CPU temperature across servers.
- ▢ **(d)** Serving web pages for cluster documentation.

Q39. What does a Slurm job state of 'NODE_FAIL' indicate?

- ▢ **(a)** The user entered an invalid password.
- ▢ **(b)** The job was terminated because the physical compute node hosting the job experienced a hardware fault (e.g., kernel panic, power failure, or hardware watchdog reset).
- ▢ **(c)** The Python interpreter found a syntax error.
- ▢ **(d)** The model reached maximum training accuracy.

Q40. Why is 'process affinity' (core pinning) essential for MPI applications running on modern NUMA supercomputers?

- ▢ **(a)** To make processes run in alphabetical order.
- ▢ **(b)** To prevent the OS kernel scheduler from migrating threads across CPU sockets, which destroys L1/L2 cache residency and forces high-latency remote NUMA memory traffic.
- ▢ **(c)** To disable Intel Turbo Boost.
- ▢ **(d)** To ensure processes can only read read-only files.

Q41. In distributed deep learning, what distinguishes 'Synchronous Data Parallelism' from 'Asynchronous Data Parallelism'?

- ▢ **(a)** Synchronous uses GPUs; asynchronous uses CPUs.
- ▢ **(b)** Synchronous waits for all workers to compute and average gradients before updating weights at a synchronization barrier; asynchronous allows workers to update weights independently without waiting.
- ▢ **(c)** Asynchronous training guarantees bit-exact mathematical reproducibility.
- ▢ **(d)** Synchronous training cannot scale beyond 2 workers.

Q42. Why does Asynchronous SGD often experience convergence degradation and lower final model accuracy at large scale?

- ▢ **(a)** Because workers forget their learning rates.
- ▢ **(b)** The Stale Gradient Problem: by the time a worker finishes calculating gradients, other workers have already updated the shared weights, causing updates to be applied to an outdated parameter state.
- ▢ **(c)** Because asynchronous training uses 8-bit integers.
- ▢ **(d)** Because Python does not support multi-threading.

Q43. In the Ring-AllReduce algorithm, how many message transfer steps are required to synchronize N workers during the Scatter-Reduce phase?

- ▢ **(a)** 1 step
- ▢ **(b)** $N - 1$ steps
- ▢ **(c)** N^2 steps
- ▢ **(d)** 2^N steps

Q44. According to the Linear Scaling Rule, if a model trained on 1 GPU with batch size 32 has a base learning rate of 0.001, what should the learning rate be when scaling to 16 GPUs with global batch size 512?

- ▢ **(a)** 0.0000625
- ▢ **(b)** 0.016

- ▢ **(c)** 0.001
- ▢ **(d)** 0.512

Q45. Why is 'Gradual Learning Rate Warmup' crucial when applying large learning rates derived from the Linear Scaling Rule?

- ▢ **(a)** To allow the cooling fans to spin up slowly.
- ▢ **(b)** Because in early training iterations, randomly initialized weights produce large, volatile gradients; applying a massive learning rate immediately causes loss explosion and divergent optimization.
- ▢ **(c)** Because Slurm kills jobs that start with high learning rates.
- ▢ **(d)** To allow the dataset to be downloaded from the network.

Q46. What is the primary cause of 'GPU Compute Starvation' in deep learning training clusters?

- ▢ **(a)** The GPU lacks floating-point registers.
- ▢ **(b)** The host CPU data loading pipeline (image decoding, data augmentation, batch shuffling) cannot prepare batches fast enough to keep GPU tensor cores continuously supplied.
- ▢ **(c)** The model has too many parameters.
- ▢ **(d)** The learning rate is set too low.

Q47. How does 'Communication-Computation Overlapping' hide network latency in distributed deep learning frameworks?

- ▢ **(a)** By executing training without using a network.
- ▢ **(b)** By launching the asynchronous AllReduce gradient synchronization for outer layers as soon as their gradients are calculated, overlapping network transfer with the backward pass of earlier layers.
- ▢ **(c)** By running data loading on the GPU.
- ▢ **(d)** By skipping the forward pass on every second batch.

Q48. Why should deep learning model checkpoints be saved strictly by rank 0 (`if hvd.rank() == 0`) in distributed cluster environments?

- ▢ **(a)** Only rank 0 has access to model weights.
- ▢ **(b)** If all distributed workers write checkpoints simultaneously to the same shared filesystem path, severe file locking conflicts, race conditions, and filesystem corruption will occur.
- ▢ **(c)** Rank 0 is the only node with internet access.
- ▢ **(d)** Other ranks will delete the model checkpoint.

Q49. What is the memory reduction factor achieved by transitioning model weights and activations from FP32 (32-bit float) to BF16 (16-bit Bfloat)?

- ▢ **(a)** $1.5\times$
- ▢ **(b)** $2\times$ (50% reduction in memory footprint)
- ▢ **(c)** $4\times$
- ▢ **(d)** $8\times$

Q50. What is a 'Distributed Straggler', and why does it disproportionately degrade synchronous distributed training performance?

- ▢ **(a)** A worker that runs faster than all other workers.
- ▢ **(b)** A single slow worker node (due to hardware throttling, network contention, or I/O stalls) that forces all other worker nodes to stall at the synchronization barrier until it catches up.
- ▢ **(c)** A network packet that gets lost.
- ▢ **(d)** A dataset sample that contains corrupted labels.